

2. Skriv kod som läser in ett heltal från användare och skriver ut Odd om talet är udda och Even om talet är jämnt.
3. Gör sanningstabeller för de booleska operatorerna and och or (d.v.s. fyll i ? i tabellen nedan):

x	y	x and y	x or y
True	True	?	?
True	False	?	?
False	True	?	?
False	False	?	?

4. Försök, utan att köra koden, utvärdera dessa fyra uttryck för alla möjliga booleska värden på x, y och z:

```
(not x) or (not y)
x and (y or z)
(x != z) and not y
x and z
```

Kör sedan koden för att bekräfta dina svar.

5. Skriv en Python-funktion som motsvarar den matematiska funktionen $f(x, y) = x/y$. Vi vill att funktionen ska säga till om man delar ett tal skilt från 0 med 0. I detta fall så ska funktionen skriva ut `The result is infinite...`, men *inte* returnera något explicit värde. Om man delar 0/0, så ska funktionen skriva ut `Indeterminate form...` och inte heller returnera något.

6. Låt:

```
x = True
y = False
```

Vilka uttryck nedan får värdet `True`? Varför?

```
(x and y) or (not x and not y)
(x or y) and (not x or not y)
x or y or not x or not y
not(not x and not (x and y))
```

7. Definiera nand, nor och xnor som Python-funktioner.

Tips: https://en.wikipedia.org/wiki/Logic_gate#Symbols.

8. Prova funktionerna nand, nor och xnor med argument som *inte* är av typen bool. Är resultaten som du väntade dig? Om inte, varför?

Kapitel 3

Listor och iteration

Det här kapitlet handlar om:

- Listor
- Iteration med **for** och **while** loopar

Listor är viktiga i programmering då de låter oss samla flera element tillsammans. Iteration handlar om att automatisera upprepning och är ett fundamentalt koncept i så kallad *imperativ* programmering. Två viktiga konstruktioner som vi ska prata om i det här kapitlet är **for** och **while** loopar. Listor och loopar är väldigt användbara tillsammans, exempelvis kan man skriva ett program som läser in tal från användaren, sparar dem i en lista och beräknar medelvärde av de inmatade talen.

3.1 Listor

Python har stöd för listor av element med olika typ. Dessa skrivs inom hakparenteser (“[” och “]”) och elementen i listan separeras med komma:

```
mylist = ['hej', 2, 'du']
```

Pythons lista kan jämföras med typen *array* i andra språk, men den är mindre beräkningseffektiv. Dock är Pythons listor mer lätthanvända. Detta är typiskt för scriptspråk: det är enklare att använda datastrukturer, på bekostnad av effektivitet.

3.1.1 Listindexering

Man kan hämta ut elementen ur en lista genom `mylist[i]` där `i` är ett heltal:

```
print(mylist[0])
print(mylist[1])
print(mylist[2])
```

```
hej
2
du
```

Viktigt: listor börjar indexeras från 0!

Man kan även använda negativa värden för att hämta ut element från slutet av listan.

```
print(mylist[-1])
print(mylist[-2])
print(mylist[-3])
```

```
du
2
hej
```

Om man använder ett index som är för stort får man ett felmeddelande:

```
print(mylist[3])
```

```
IndexError: list index out of range
```

Fråga: vad händer om man indexerar med ett för stort negativt värde (t.ex. `mylist[-4]`)?

Man kan lägga till element i slutet av en lista med `append`.

```
mylist.append(True)
print(mylist)
```

```
['hej', 2, 'du', True]
```

Man kan enkelt tilldela ett nytt värde till ett element i en lista.

```
mylist[1] = 5
print(mylist)
```

```
['hej', 5, 'du', True]
```

Vi kan även göra detta genom att indexera från slutet av listan:

```
mylist[-2] *= 3
print(mylist)
```

```
['hej', 5, 'dududu', True]
```

Man kan hämta ut alla element med index $i < j$ genom `mylist[i:j]`. Detta kallas för *slicing*.

```
print(mylist[0:1])
print(mylist[0:2])
print(mylist[1:3])
print(mylist[1:4])
```

```
['hej']
['hej', 5]
[5, 'dududu']
[5, 'dududu', True]
```

Man kan ändra steglängden i slicingen till `k` genom `mylist[i:j:k]`:

```
print(mylist[0:4:2])
print(mylist[1:4:2])
```

```
['hej', 'dududu']  
[5, True]
```

3.1.2 Listfunktioner

Man kan addera listor och multiplicera dem med tal för att upprepa dem:

```
print(mylist + mylist)  
print(2 * mylist)
```

```
['hej', 5, 'dududu', True, 'hej', 5, 'dududu', True]  
['hej', 5, 'dududu', True, 'hej', 5, 'dududu', True]
```

Längden av en lista räknas ut med len:

```
print(len(mylist))  
print(len(5 * mylist + mylist))
```

```
4  
24
```

En väldigt användbar funktion är **in** som testar om ett element finns i en lista:

```
print(5 in mylist)  
print(False in mylist)
```

```
True  
False
```

Jämför med $x \in S$ från matematiken som säger att x finns i mängden S . Funktionen **in** fungerar även på strängar:

```
print('gg' in 'eggs')
```

```
True
```

Det finns även **not in** vilket motsvarar $x \notin S$ från matematiken:

```
print(5 not in mylist)  
print(False not in mylist)
```

```
False  
True
```

Man beräknar minsta och största element ur en lista med min och max:

```
print(min([2,5,1,-2,100]))  
print(max([2,5,1,-2,100]))
```

```
-2  
100
```

Man kan ta reda på vilket index ett element finns på med hjälp av index:

```
print(mylist.index(True))
```

3

Försöker man hitta index för något som inte finns i listan får man ett felmeddelande:

```
print(mylist.index(42))
```

```
ValueError: 42 is not in list
```

Man kan räkna antalet gånger ett element finns med i en lista med count:

```
mylist.append(5)
print(mylist.count(5))
print(mylist.count(42))
```

2
0

Vi kan ta bort element på ett specifikt index eller en hel slice med **del**:

```
print(mylist)
del mylist[2]
print(mylist)
del mylist[0:2]
print(mylist)
```

```
['hej', 5, 'dududu', True, 5]
['hej', 5, True, 5]
[True, 5]
```

Detta är samma som `mylist[i:j] = []`. Man kan även använda **del** `mylist[i:j:k]` för att få en annan steglängd.

Vi kan även ta bort elementet på index i och returnera det med pop:

```
mylist = ['hej', 5, 'dududu', True, 5]
print(mylist)
x = mylist.pop(2)
print(mylist)
print(x)
```

```
['hej', 5, 'dududu', True, 5]
['hej', 5, True, 5]
dududu
```

Detta fungerar även med negativa värden (dvs, genom att man indexerar från slutet). Ger man inget argument till pop så är det samma som `pop(-1)`, dvs sista elementet tas bort.

```
x = mylist.pop()
print(mylist)
print(x)
```

```
['hej', 5, True]
5
```

Man kan stoppa in ett element x på ett givet index i i en lista med `insert(i, x)`:

```
mylist.insert(1, 'du')
print(mylist)
```

```
['hej', 'du', 5, True]
```

För att ta bort första elementet som är lika med något x använder man `remove(x)`:

```
mylist.append(5)
print(mylist)
mylist.remove(5)
print(mylist)
```

```
['hej', 'du', 5, True, 5]
['hej', 'du', True, 5]
```

Slutligen kan man använda `reverse` för att vända på en lista:

```
mylist.reverse()
print(mylist)
```

```
[5, True, 'du', 'hej']
```

3.1.2.1 Sammanfattning av användbara listfunktioner

- `mylist.append(x)`: lägg till x i slutet av mylist.
- `len(mylist)`: beräkna längden av mylist.
- `x in mylist`: testa om x finns i mylist (returnerar en bool).
- `x not in mylist`: testa om x inte finns i mylist (returnerar en bool).
- `min(mylist)` och `max(mylist)`: hämta ut minsta respektive största element i mylist.
- `mylist.index(x)`: räkna ut första index där x finns i mylist.
- `mylist.count(x)`: räkna hur många gånger x finns i mylist.
- `del mylist[i]`: ta bort elementet på index i ur mylist (utan att returnera det).
- `mylist.pop(i)`: ta bort elementet på index i ur mylist (och returnera det).
- `mylist.insert(i,x)`: sätt in x på index i i mylist.
- `mylist.remove(x)`: ta bort första förekomsten av x ur mylist.
- `mylist.reverse()`: vänd mylist baklänges.

För mer listfunktioner och detaljer om listor se: <https://docs.python.org/3/tutorial/datastructures.html#more-on-lists>

<https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range>

<https://docs.python.org/3/library/stdtypes.html#lists>

3.1.3 Praktisk funktion för att skapa listor: range

Ett sätt att skapa heltalslistor är att använda `range`:

```
print(list(range(1,4)))
print(list(range(3)))      # samma som range(0,3)
print(list(range(4,2)))
print(list(range(0,10,2)))
print(list(range(5,0,-1))) # iterera baklänges
```

```
[1, 2, 3]
[0, 1, 2]
[]
[0, 2, 4, 6, 8]
[5, 4, 3, 2, 1]
```

Obs: vi måste konvertera resultatet till en lista med hjälp av `list(...)` då `range` inte ger en lista direkt, utan ett “löfte” om en lista. Dvs, `range` returnerar ett objekt som skapar en lista vartefter man behöver en. Detta beteende kallas för *lat evaluering* och är smidigt då man inte behöver skapa hela listan på en gång. Detta har fördelen då hela listan inte behöver sparas i minnet på datorn på en gång vilket kan bli långsamt för stora listor. Mer om detta senare.

Om man försöker skriva ut `range(0, 10)` utan att konvertera till en lista händer följande:

```
print(range(0, 10))
```

```
range(0, 10)
```

3.1.4 Listors egenskaper

Listor har många styrkor, bland annat:

1. Enkla och effektiva att iterera över (mer om detta i nästa stycken).
2. Enkla att utöka.
3. Omedelbar åtkomst om man har index.
4. Listelementen kan ha olika typ.

Egenskap 3 är speciell i Python. Andra språk, som t.ex. C och Java, skiljer på *array* och *lista*. En array är som en vektor och har en fix storlek. I dessa språk är listor istället en speciell datastruktur där elementen är “länkade” så att man bara har omedelbar tillgång till det första (och kanske sista) elementet. Det gör att man inte automatiskt får omedelbar tillgång till alla element. Men i Python har man alltså direkt åtkomst till alla element genom listindexering vilket gör att dom påminner lite om arrayer.

En svaghet med listor är att de är ineffektiva att *leta i* (dvs operatören `in` kan vara långsam på stora listor). En annan svaghet är att oväntade saker kan hända om man har en lista som innehåller element av olika typ och försöker hämta ut största/minsta värde (se uppgift 2).

3.2 for loopar

for används för att iterera över uppräknings av olika slag, framförallt listor. Dessa uppräknings kallas *sekvenser* i Python och även strängar är sekvenser (av tecken).

```
for x in [1, 2, 3]:
    print(x)
```

```
1
2
3
```

Obs: notera att `print` satsen är indragen! Allt som kommer efter **for** och som är indenterat (med lika många mellanslag) kommer att köras i loopen. Som vi såg i förra kapitlet är Python indenteringskänsligt, dvs det är viktigt att man indenterar kodblock lika mycket och detta gäller även för **for** loopar. Detta

skiljer sig från språk som C och Java där man istället grupperar kodblock inom måsvingar (dvs “{” och “}”) och rader avslutas med semikolon “;”. Skulle man ha indenterat något fel i Python får man felmeddelandet:

```
IndentationError: expected an indented block
```

Ett kodblock kan innehålla flera instruktioner:

```
s = 'A number: '  
  
for x in [0,1,2]:  
    print(x)  
    s += str(x)  
  
print(s)
```

```
0  
1  
2  
A number: 012
```

Vi kan även iterera över en lista som ligger i en variabel:

```
mylist = [112, 857, 'hej']  
  
for x in mylist:  
    print(x)
```

```
112  
857  
hej
```

Eftersom strängar också är sekvenser kan man lätt iterera över bokstäverna i en sträng:

```
for c in 'DA2005':  
    print(2*c)
```

```
DD  
AA  
22  
00  
00  
55
```

3.2.1 Funktion som returnerar konsonanterna i en sträng

Låt oss nu implementera en lite mer komplicerad funktion som returnerar en strängs konsonanter. Innan man börjar skriva koden bör man fundera hur funktionen ska fungera. Givet en sträng s så ska den iterera över alla bokstäver och testa om de är konsonanter. Om bokstaven var det så ska den läggas till en sträng med alla konsonanter vi hittat i s och annars ska inget hända. Detta kan implementeras på följande sätt:

```
def consonants(s):
    cs = 'BCDFGHJKLMNPQRSTVWXZbcdfghjklmnpqrstvwxyz'

    out = ''

    for c in s:
        if c in cs:
            out += c

    return out

print(consonants('Hey Anders!'))
```

Hndrs

Notera hur vi definierar en variabel `out` vilken används för att spara de konsonanter vi hittat i `s`. Detta är ett väldigt vanligt sätt att programmera på och något vi kommer se mer av i kursen.

3.2.2 Dela upp en sträng i ord: `split`

Låt oss skriva en funktion som tar en sträng och bryter upp den i smådelar när man hittar ett blanksteg som separator. Det vill säga vi ska skriva en funktion `split(s)` som tar in en sträng `s` och returnerar alla ord i strängen. Detta kan göras genom att man har två variabler, en med de ord vi hittat och en med alla bokstäver i ordet vi håller på att bygga upp. För varje bokstav i `s` testas om den är ett mellanslag eller inte. Om bokstaven är ett mellanslag så lägger vi det inlästa ordet i listan, annars så lägger vi till bokstaven till ordet och tittar på nästa bokstav.

```
def split(s):
    out = []
    term = ''

    for c in s:
        if c == ' ':
            out.append(term)
            term = ''
        else:
            term += c

    # add the final word unless it is empty
    if term != '':
        out.append(term)

    return out

print(split('Hey Anders! How are you?'))
```

['Hey', 'Anders!', 'How', 'are', 'you?']

Säg nu att vi vill generalisera detta till många olika möjliga separatorer, men att mellanslag ska vara standardvärdet, då skriver vi istället följande kod:

```
def split(s, sep = ' '):
    res = []
    term = ''
    for c in s:
        if c in sep:
            res.append(term)
            term = ''
        else:
            term += c

    if term != '':
        res.append(term)

    return res

print(split('Hey Anders! How are you?'))
print(split('Hey Anders! How are you?', '!'))
```

```
['Hey', 'Anders!', 'How', 'are', 'you?']
['Hey Anders', ' How are you?']
```

Vi kan nu alltså anropa funktionen med ett *eller* två argument.

Obs: det finns redan en variant på `split` inbyggd i Python!

3.2.3 Flödeskontroll: `pass`, `continue`, `break` och `return`

Man kan kontrollera och avsluta iteration med `pass`, `continue`, `break` och `return`.

Den enklaste av dessa är `pass` som inte gör någonting:

```
for i in range(4):
    if i == 1:
        pass
        print('passed 1')
    else:
        print(i)
```

```
0
passed 1
2
3
```

Obs: `pass` operationen kan verka oanvändbar då den inte gör någonting, men ett vanligt användningsområde är när man strukturerar upp sitt program men inte har skrivit klart allt. Man kan till exempel definiera en funktion där kroppen bara är `pass` eller ha en `if` sats där något fall inte är skrivet än. Detta gör att man kan testa all annan kod i programmet och sedan skriva klart de delar där man använt `pass` efter att man försäkrat sig om att allt annat fungerar.

En `continue` instruktion avslutar nuvarande iteration av loopen och går vidare till nästa:

```
for i in range(4):
    if i == 1:
        continue
    print('passed 1')
    else:
        print(i)
```

```
0
2
3
```

En **break** instruktion avslutar loopen helt:

```
for i in range(4):
    if i == 2:
        break
    else:
        print(i)
```

```
0
1
```

Slutligen, **return** används i en loop när man vill returnera något från en funktion. Till exempel:

```
def first_even_number(l):
    for number in l:
        if number % 2 == 0:
            return number

print(first_even_number([1,45,24,9]))
```

```
24
```

3.2.4 Pythons for är annorlunda

Det kan vara bra att vara medveten om att Pythons **for** loop är mer generell än i de flesta andra språk. Till exempel förväntas looperna gå över heltalsvärden i många äldre språk. Pythons konstruktion kan sägas vara modern: den bygger på att man konstruerar en datastruktur som stödjer iteration. Det är fullt möjligt, och inte så svårt, att skriva sina egna itererbara datatyper för speciella behov. Som vi sett går det att iterera över både listor och strängar, men vi kommer att se att även andra typer i Python stödjer det.

3.2.5 Ett vanligt problem med listor i Python: muterbarhet

Ett vanligt problem med listor i Python som vi kommer se flera gånger i kursen är att man kan råka ändra på listan som man itererar över utan att vilja det. Detta är en väldigt vanlig källa till buggar, både för nybörjare och mer erfarna programmerare.

Låt oss säga att vi vill köra följande program som är tänkt att lägga in de ord som är längre än 6 bokstäver längst fram i listan:

```
animals = ['cat', 'dog', 'tortoise', 'rabbit']

for x in animals:
```

```

if len(x) > 6:
    animals.insert(0, x)

print(animals)

```

Men kör man detta kommer Python bara loopa och inte komma ur loopen! För att avsluta loopen trycker man CTRL-C. Problemet är att vi ändrar på `animals` listan samtidigt som vi loopar över den, så **for** loopen kommer aldrig avslutas.

För att undvika detta bör man itererar över en *kopia* av listan. Vi kan göra detta genom att använda `.copy()`:

```

animals = ['cat', 'dog', 'tortoise', 'rabbit']

for x in animals.copy():
    if len(x) > 6:
        animals.insert(0, x)

print(animals)

['tortoise', 'cat', 'dog', 'tortoise', 'rabbit']

```

Här används `animals.copy()`, istället för bara `animals`, så att vi får en kopia av listan och undviker den oändliga loopen.

Detta problem med listor kan komma upp när man programmerar och inte är försiktig med att kopierar saker. Detta kan leda till väldigt mystiska buggar som den oändliga loopen ovan. Det som gör att Pythons listor beter sig så här är att de är *muterbara*. Vi kommer prata mer om muterbarhet senare i kursen, men i dagsläget behöver ni bara komma ihåg att man kan behöva använda `.copy()` på en listvariabel för att undvika att råka ändra på den när man inte vill det.

3.3 while loopar

Ibland finns det inte någon självklar struktur att iterera över, men man vill ändå repetera ett kodblock flera gånger. Då finns **while** loopen. Den fungerar så att man kör ett kodblock tills något booleskt test är uppfyllt. Om vi kör följande exempel:

```

i = 10

while i > 0:
    print(i)
    i -= 1

print('Done!')

```

```

10
9
8
7
6
5

```

```
4
3
2
1
Done!
```

Ett vanlig användningsområde för **while** loopar är att loopa tills man fått rätt indata från en användare. Programmet nedan kommer läsa in indata från användaren tills man skriver exit:

```
loop = True

while loop:
    print("Write 'exit' to quit")
    answer = input('Write something: ')
    if answer == 'exit':
        print('Goodbye!')
        loop = False
    else:
        print('Let us keep going!')
```

```
Write 'exit' to quit
Write something: hey
Let us keep going!
Write 'exit' to quit
Write something: ho
Let us keep going!
Write 'exit' to quit
Write something: exit
Goodbye!
```

3.3.1 while loopar över listor

Listor är speciella och kan användas som booleska uttryck. Den tomma listan motsvarar False och alla andra listor motsvarar True.

```
list = []

if list:
    print('Not empty!')
else:
    print('Empty!')

list2 = ['Hey']

if list2:
    print('Not empty!')
else:
    print('Empty!')
```

```
Empty!  
Not empty!
```

Denna egenhet är till för följande *idiom* där man loopar över en lista och plockar bort dess element:

```
mylist = ['Ho', 'ho', 'ho!']  
  
while mylist:  
    x = mylist.pop()  
    print(x)  
  
print(mylist)
```

```
ho!  
ho  
Ho  
[]
```

När loopen är klar är listan tom och man behöver alltså inte jämföra listan med []. Detta är alltså ett till exempel där Python har förändrat en lista i en loop.

3.3.2 Ta bort blanksteg i slutet av en sträng

Så här kan man ta bort blanksteg i slutet av en sträng med hjälp av en **while** loop och slicing:

```
def remove_trailing_space(s):  
    i = len(s)  
    while i > 0 and s[i-1] == ' ':  
        i -= 1  
    return s[0:i]  
  
print(remove_trailing_space('SU  '))
```

```
SU
```

Funktionen fungerar på så sätt att man börjar med en räknare *i* som sätts till längden på listan. Sen räknar vi hur många tecken från slutet av listan som är blanksteg och när vi är klara returnerar vi en slice av de ursprungliga listan där vi tagit bort alla blanksteg i slutet.

3.3.3 Största gemensamma delare: GCD

Den *största gemensamma delaren* (eng.: *greatest common divisor*, GCD) av två tal är det största heltal som delar dem. Exempelvis är 3 GCD av 15 och 12.

Euklides algoritm för att beräkna GCD för två tal *a* och *b* lyder så här:

- Antag $a \geq b$,
- låt *r* vara resten av heltalsdivision $a // b$,
- om $r == 0$, då är $\text{GCD}(a, b) = b$,
- annars kan man använda att $\text{GCD}(a, b) = \text{GCD}(b, r)$.

Lägg märke till att det är en villkorad iteration, men ingen tydlig struktur att iterera över. Så vi bör använda en **while** loop för att implementera den:

```
def GCD(a,b):
    if b > a:
        print('a must be bigger than b in GCD(a,b)')
        return None

    r = a % b
    while r != 0:
        a = b
        b = r
        r = a % b
    return b

print('GCD: ' + str(GCD(15,12)) + '\n')
print('GCD: ' + str(GCD(12,15)))
```

GCD: 3

b must be bigger than a in GCD(a,b)
GCD: None

3.4 Uppgifter

1. Vad är resultatet av `[1,2] + ['a', 'b']`?
2. Vad händer om man använder `min/max` på listor med element av olika typ?
3. Skriv klart funktionen

```
def vowels(s):
    vs = 'AEIOUYaeiouy'
    pass
```

vilken tar in en sträng `s` och returnerar dess vokaler.

4. Modifiera funktionen i uppgift 3 så att den istället bara returnerar konsonanter utan att ändra på strängen `vs`. Skriv sen en funktion `vowels_or_consonants` med en extra parameter vilken bestämmer om vokaler eller konsonanter ska returneras. Parametern ska ha ett standardvärde så att vokaler returneras om användaren inte kallar på funktionen med två argument.
5. Vad händer om man inte har blocket med `if` term `!= ''` i funktionen `split(s)` i avsnitt avsnitt 3.2.2? Hitta på ett exempel med en sträng `s` där utdata blir fel om man inte har det. Vidare, hur beter sig vår funktion `split` för strängar med flera mellanslag i rad? Skriv sen en korrekt version av funktionen.
6. Testa följande kodsnuitt:

```
list = [1,2,3]
list2 = list
list2.reverse()
print(list)
```

Fungerar den som ni trodde? Varför inte? Hur kan man fixa det?

7. Skriv om koden

```
mylist = ['Ho', 'ho', 'ho!']

while mylist:
    x = mylist.pop()
    print(x)

print(mylist)
```

så att listan `mylist` inte är tom när man är klar.

Tips: använd `.copy()`. Hur kan man göra om man inte får använda `copy`?

8. Testa att använda **continue**, **break** och **pass** i **while** loopen nedan. Vad händer?

```
n = 10
while n > 0:
    n -= 1
    if n == 7:
        # test continue, break or pass here
    print(n)
```

9. Skriv en **while** loop som läser in tal från användaren och sparar dem i en lista. Detta ska göras tills användaren skriver in 0 och då ska loopen avslutas och listan skrivas ut.

10. Skriv funktionen

```
def naive_gcd(a,b):
    pass
```

som beräknar GCD av `a` och `b` genom att testa alla tal mellan 1 och `min(a,b)` för att se vilket som är det största talet som delar båda.

11. Skriv funktionen `portkod()` som läser in en siffra (1–9) i taget från användaren. Efter varje siffra är inläst ska funktionen testa om de fyra senaste siffrorna matchar med korrekt kod. Om korrekt kod ges ska funktionen skriva ut `Door unlocked!` och returnera `True`. Välj vad som är korrekt portkod själv, t.ex. 1337.

Tanketips: Vilken loop ska vi använda?

12. Om vi har `lst = [1, 2, 3]` så kan man stoppa in ett element mellan 1:an och 2:an med instruktionen `lst.insert(1, 'x')`. Resultatet är att listan blir `[1, 'x', 2, 3]`.

Ett annat men ekvivalent sätt är att skriva `lst[1:1] = ['x']`.

- Vad är det som händer i den instruktionen?
- Vilket sätt tycker du är tydligast?